

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



REPORT

COURSE: DISTRIBUTED SYSTEMS

STREAMING LAKEHOUSE DATA PLATFORM

SUPERVISOR(s): PhD. NGUYEN QUANG HUNG

STUDENT 1: NGO QUANG HIEU - 2311015

STUDENT 2: MAI THANH HOANG - 2311048

STUDENT 3: LE VU MINH HIEU - 2310953

—o0o—

HO CHI MINH CITY, Sep 2026

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of PhD. NGUYEN QUANG HUNG at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2027

Authors,

Ngo Quang Hieu

Acknowledgement

We would like to express my appreciation to Dr. NGUYEN QUANG HUNG for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from Nguyen Thanh for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

This thesis presents *Streamify*, a project built to learn, hands-on, how a modern streaming platform works and to apply distributed-systems concepts to a real system, rather than studying them in the abstract. A continuous stream of synthetic music-streaming events is carried end to end: ingested with ordering and fault-tolerance guarantees, processed continuously as it arrives rather than on a batch schedule, landed into a storage layer that supports both transactional updates and analytical queries, transformed into an analytics-ready model, and visualized and monitored throughout – all under a small, fixed compute budget rather than assumed elastic scale.

Building this meant confronting, and resolving, the concrete distributed-systems problems that appear once a platform has more than one producer and consumer of the same data: partitioning and replication for ordered, fault-tolerant ingestion; checkpointed, exactly-once processing so a crash never silently loses or duplicates an event; a storage layer that serves frequent small writes and large analytical reads without one starving the other; and observability that shows whether the system is keeping up with its input, not just whether it is running.

Every major technology choice was made deliberately against concrete alternatives and the constraints of a single-machine deployment, rather than assumed as a given (Chapter 2). This thesis documents that reasoning alongside the resulting architecture (Chapter 4) and what building and operating the system revealed about how these distributed-systems concepts behave in practice (Chapter 5).

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	2
1.4	Report Structure	3
2	Related Work and Background	4
2.1	Why Real-Time Processing Matters	4
2.2	Message Queues and Publish-Subscribe Messaging	6
2.3	Batch and Streaming Processing Engines	7
2.4	Storage layer & Open Table Formats	8
2.5	Medallion Layering	10
3	Problem Statement and Challenges	11
3.1	The Problem Before Streaming Lakehouses	11
3.1.1	Latency of Batch ETL	11
3.1.2	Lambda-Architecture Complexity	11
3.1.3	No Serving Layer on the Data Lake	12
3.1.4	Storage and Compute Coupled in the Warehouse	12
3.2	Technical Challenges	12
3.2.1	Streaming Volume and Velocity	13
3.2.2	Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster	13
3.2.3	Computing and Partitioning on a Small, Shared Spark Cluster .	13

4	Proposed Solution	15
4.1	Architecture Overview	15
4.2	Storage Layer	17
4.3	Ingestion Layer	17
4.4	Stream & Batch Processing Layer	18
4.5	Transformation Layer	21
4.6	Serving Layer	23
4.7	Monitoring Layer	24
4.8	Deployment & Environment	25
5	Conclusion	26
5.1	Results Achieved	26
5.2	Limitations	27
5.3	Future Directions	27
	References	27

List of Tables

4.1	Technology stack per pipeline layer.	16
-----	--	----

List of Figures

1	Producers and consumers decoupled through a publish-subscribe message queue.	6
2	Batch processing versus streaming (event-at-a-time and micro-batching).	7
3	Object storage versus the table abstraction built on top of it.	8
4	Overview of an open table format, contrasting copy-on-write and merge-on-read.	9
5	Medallion architecture: successive bronze-to-gold refinement layers. . .	10
6	Streamify architecture overview	15
7	Kafka internal architecture: 2-node KRaft cluster, 3 partitions per topic, replication factor 2.	17
8	Spark cluster architecture: driver, master, and worker/executor processes.	18
9	Spark Structured Streaming’s micro-batch model: an unbounded stream is sliced into small batches, each run as an ordinary Spark batch job. . .	19
10	Star schema: <code>fact_streams</code> and its five dimension tables.	21
11	dbt DAG showing the dependency between bronze sources, the star schema, and <code>wide_streams</code>	22
12	Superset results dashboard, built on top of <code>wide_streams</code> and queried through the Spark Thrift Server.	23
13	Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.	24
14	Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.	24

Chapter 1

Introduction

In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.

1.1 Motivation

Modern data platforms increasingly need to react to events as they happen rather than waiting for the next batch load. A music-streaming service, for example, wants to know within seconds which songs are trending, whether a user’s session just dropped, or whether ingestion is falling behind – questions a traditional overnight ETL job cannot answer. Building and operating such a platform correctly requires more than a single tool: it requires a message queue that can buffer and replay events, a stream-processing engine that can keep up with continuous arrival, a storage layer that supports transactional updates on top of cheap object storage, a transformation layer that turns raw events into trustworthy analytics tables, and a way to observe the whole system’s health while it runs.

This project, *Streamify*, was motivated by the goal of building and demonstrating exactly such an end-to-end pipeline, using synthetic but realistic music-streaming events generated by Eventsim, rather than studying any single one of these components in isolation. Each stage of the pipeline – ingestion, stream processing, lakehouse storage, transformation, and visualization – was chosen and configured to work together as a single reproducible system, so that the trade-offs made at each stage (Chapter 2) could be

evaluated in the context of the whole pipeline rather than as isolated technology choices.

A further motivation was to explore these technologies under realistic resource constraints. Rather than assuming unlimited cloud compute, the project deliberately targets a small, single-machine deployment (Section 4.8), so that decisions such as Kafka cluster sizing and Spark cluster sizing (Section 4.8) had to be made under the same resource pressure a small team or a proof-of-concept deployment would face in practice.

1.2 Goals

The goals of this project are:

- **Build a working, end-to-end streaming lakehouse pipeline, and learn how a modern streaming platform works.** Take synthetic music-streaming events from generation (Eventsim) through ingestion (Kafka), stream processing (Spark Structured Streaming), lakehouse storage (Hudi), transformation (dbt), and visualization (Superset), as a single reproducible system rather than isolated proofs of concept.
- **Apply the distributed-systems concepts from the course to a real system.** Exercise partitioning, replication, checkpointed exactly-once processing, and consumer-lag monitoring, so the pipeline shows the behaviors that distinguish a streaming design from a periodic batch job (Section 2.4).
- **Make the whole system observable and reproducible.** Provide Prometheus/Grafana monitoring of Kafka lag and Spark resource usage, and deploy the entire stack with a single Docker Compose command.

1.3 Scope

This project covers the full path from synthetic event generation to dashboarding: Eventsim event generation, Kafka ingestion, Spark Structured Streaming processing into Hudi bronze tables, dbt transformation into a star schema and wide table, Superset dashboarding, and Prometheus/Grafana monitoring of the pipeline's own health (Chapter 4). Within this path, the scope also includes the distributed-systems concerns the course

covers – partitioning, replication, checkpointed exactly-once processing, and consumer-lag monitoring (Section 2.4) – rather than just wiring the tools together. All of these components are deployed and demonstrated together via Docker Compose on a single machine.

The following are explicitly out of scope:

- **Production security hardening.** Authentication, authorization, network isolation, secrets management, and TLS between services are not addressed; the stack is configured for local development and demonstration, not for exposure to untrusted networks.
- **Multi-region or high-availability production deployment.** The pipeline targets a single-machine, single-region Docker Compose deployment (Section 4.8); cross-region replication, multi-datacenter failover, and elastic autoscaling of the Kafka or Spark clusters are not implemented.
- **Real user data.** All events are generated by Eventsim against the Million Song Dataset’s 10,000-song subset (Chapter 3) and are entirely synthetic; the pipeline has not been run against real production traffic or real user data.

1.4 Report Structure

The remainder of this report is organized as follows:

- **Chapter 2, Related Work and Background Theory,** first covers the distributed-systems concepts behind the pipeline – event streaming and pub-sub messaging, Spark Structured Streaming’s micro-batch model, Hudi’s table formats, and data modeling.
- **Chapter 3, Problem Statement and Challenges,** describes the dataset used and the technical challenges of building a fault-tolerant streaming pipeline on a small, shared compute cluster.
- **Chapter 4, Proposed Solution,** presents the system architecture, the implementation workflow, and the cluster/environment setup.

- **Chapter 5, Conclusion**, summarizes the results achieved, current limitations, and directions for future work.

Chapter 2

Related Work and Background

This section introduces the general concepts behind a streaming lakehouse – message queues, stream processing engines, open table formats, and dimensional modeling – independent of any one product. Chapter 4 then explains which concrete tool implements each concept in this project, and why that tool was chosen over its alternatives.

2.1 Why Real-Time Processing Matters

A modern platform such as a music-streaming service does not produce data in daily batches: it produces a continuous stream of small events – a song played, a page viewed, a login – the instant a user acts. Streamify, the project this report builds, simulates exactly this kind of platform with Eventsim, which generates synthetic “song played”, “page view”, and “authentication” events for a fake music website at a steady, ongoing rate.

In a traditional batch-processing setup, these events would simply accumulate in a source system and be pulled into the warehouse on a fixed schedule – once an hour, once a day. Between runs, every dashboard and every downstream decision is working from data that is, on average, half of that interval out of date; right before the next run, it is a full interval out of date. For a platform that wants to react to what users are doing now – surfacing trending songs, flagging a spike in failed logins, feeding a recommendation engine – that lag is the problem: the data is correct, just always late.

Real-time (streaming) processing closes this gap: instead of waiting for a scheduled run, each event is picked up and made queryable within seconds to minutes of happening,

so dashboards and downstream models reflect what is happening now rather than what happened last interval. In Streamify, this means a play event lands in its Kafka topic within milliseconds of Eventsim emitting it, and is queryable as a Hudi bronze row one Spark Structured Streaming micro-batch later (Figure 2) – typically a few seconds – rather than at the next hourly or daily load.

Doing this reliably, and not just quickly, is harder than it sounds once the platform has more than one producer and more than one consumer of the same data. Four concrete problems show up:

- **Ordering.** Events from different partitions of the same topic can arrive and be processed out of order relative to each other.
- **Fault tolerance.** Any component in the chain – broker, processing job, storage layer – can crash mid-stream and must be able to resume without losing data.
- **Exactly-once delivery.** On a retry after a crash, the same event must not be silently duplicated or dropped.
- **Schema drift.** Upstream event fields change shape over time (added, renamed, or retyped), and downstream tables must tolerate that without breaking.
- **Read/write contention.** The storage layer must serve a constant stream of small writes and large analytical reads at the same time, without one starving the other.

These are the distributed-systems problems that the rest of this section’s concepts exist to solve: a *message queue* for ordered, fault-tolerant ingestion; a *stream processing engine* for continuous, exactly-once processing; and an *open table format* for a storage layer that supports both frequent streaming writes and fast analytical reads.

2.2 Message Queues and Publish-Subscribe Messaging

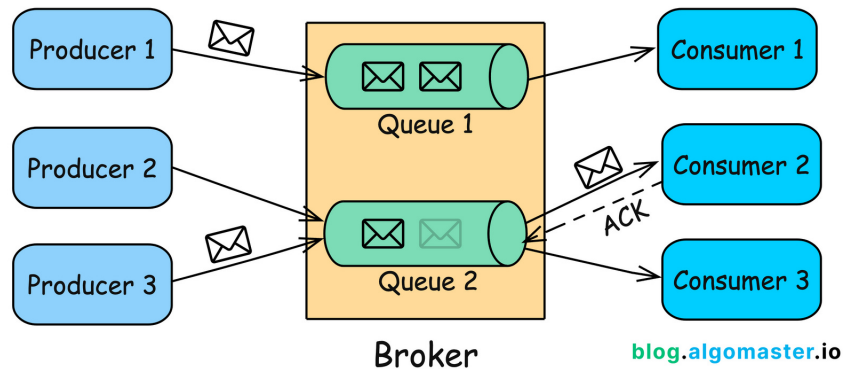


Figure 1: Producers and consumers decoupled through a publish-subscribe message queue.

Event streaming treats data as an unbounded sequence of immutable events (e.g. “user played a song”) rather than as static rows in a table. A *message queue* built for this pattern is typically a log-based, publish-subscribe (pub-sub) broker, shown in Figure 1: producers append events to a durable, ordered log rather than pushing directly to a fixed receiver, and publish them under named *topics*; one or more consumers subscribe to the topics they need and read the log independently, at their own pace. This decouples producers from consumers in both time and number – a producer does not need a consumer to be running or fast enough to keep up, and any number of independent consumers can read the same retained events without the broker deleting them on delivery.

2.3 Batch and Streaming Processing Engines

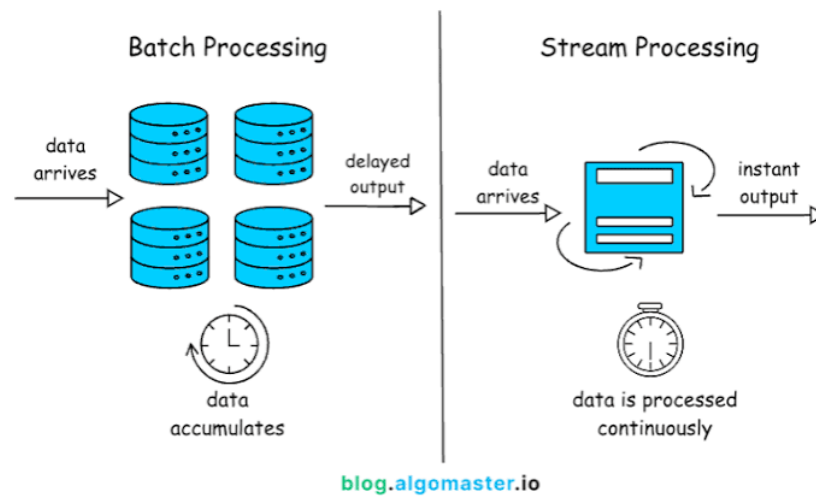


Figure 2: Batch processing versus streaming (event-at-a-time and micro-batching).

A *batch processing engine* operates on a bounded, complete dataset that is already at rest: it reads the full input, runs the job once end to end, produces its output, and stops until the next scheduled run.

A *stream processing engine* instead operates on an unbounded dataset that keeps arriving: it stays running indefinitely, continuously reading new events from a message queue, applying the same kind of transformation logic (filtering, aggregation, joins) to them, and writing results incrementally as they are produced, rather than waiting for a full batch to accumulate first. Figure 2 contrasts the two.

A stream processing engine can still process its unbounded input using either of two execution models:

- **Event-at-a-time.** Each event is processed as soon as it arrives, giving the lowest possible per-event latency.
- **Micro-batching.** The engine groups whatever new data has arrived since the last run into a small, discrete batch job on a fixed trigger interval, and processes that batch with the same execution engine used for ordinary batch queries. This reuses a mature batch engine's query planning and fault tolerance, at the cost of end-to-end latency being bounded by the trigger interval rather than being instantaneous.

Either execution model still reads its input positions (offsets) from the upstream message queue, and both keep the engine running continuously – the distinction from batch processing is not how the input is chopped up, but that the engine never stops waiting for the next event.

2.4 Storage layer & Open Table Formats

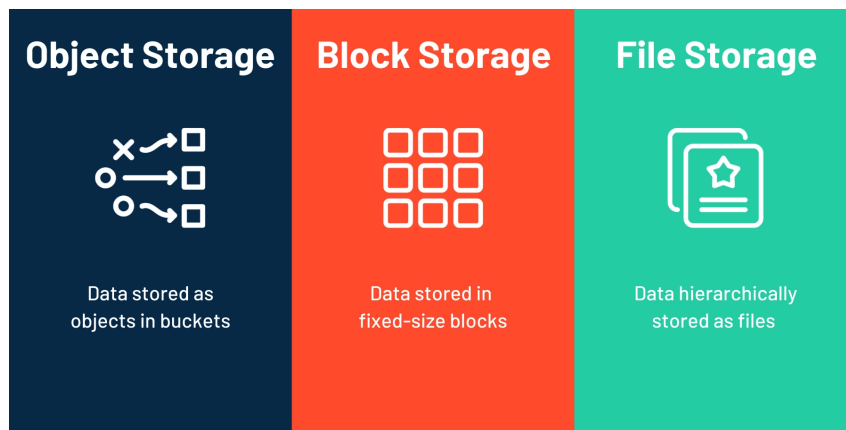


Figure 3: Object storage versus the table abstraction built on top of it.

As Figure 3 shows, a lakehouse’s underlying storage is one of three kinds:

- **Object storage.** Data is stored as whole objects in flat buckets, addressed by key rather than by a filesystem path (e.g. S3, MinIO). This is the layer a lakehouse is built on.
- **Block storage.** Data is stored in fixed-size blocks that an operating system assembles into a filesystem, as used by a traditional database’s local disk.
- **File storage.** Data is stored hierarchically as files in a shared, path-addressed filesystem (e.g. HDFS, NFS).

A lakehouse is built on object storage because it is cheap, horizontally scalable, and durable, but plain object storage is not a database: it does not provide row-level insert/update/delete semantics, ACID transactions, or time-travel queries – it only stores and returns whole objects. An *open table format* adds those capabilities on top of plain

files in object storage, so a lake of files behaves like a table rather than an unmanaged pile of Parquet.

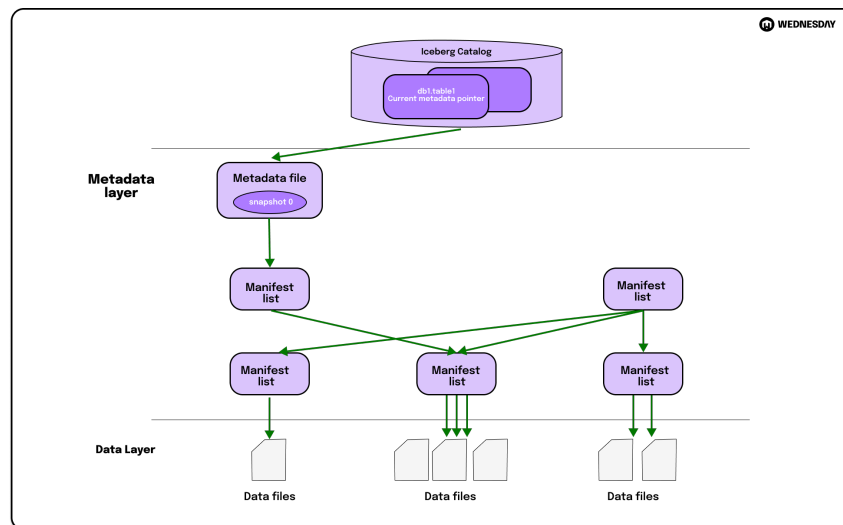


Figure 4: Overview of an open table format, contrasting copy-on-write and merge-on-read.

Open table formats typically support two write strategies that trade off write cost against read cost, as summarized in Figure 4:

- **COPY_ON_WRITE (COW).** Every write rewrites the affected data files as new versioned files. Reads are fast (plain columnar files, no merge step), but writes are more expensive because entire files are rewritten on update.
- **MERGE_ON_READ (MOR).** Writes are appended to a compact row-based delta log next to the existing base files; the two are merged only at read time (or during a background compaction job). Writes are cheap and low-latency, at the cost of extra work on read (or a periodic compaction step) to materialize the merged view.

MOR suits high-frequency streaming appends, where write latency matters most; COW suits tables that are rebuilt in full on a schedule and then read many times, where read performance matters most.

2.5 Medallion Layering



Figure 5: Medallion architecture: successive bronze-to-gold refinement layers.

The medallion architecture, shown in Figure 5, organizes a lakehouse into three successive layers of increasing refinement: bronze, silver, and gold.

- **Bronze** tables hold raw or lightly-cleaned data close to its source form, written as-is by the ingestion layer.
- **Silver** tables hold data that has been cleaned, deduplicated, and conformed to a consistent schema, but not yet aggregated or reshaped for a specific consumer.
- **Gold** tables hold data that has been modeled and aggregated into the shape consumers (BI tools, dashboards) actually query, typically built by a transformation layer reading from silver (or, when a project skips silver, directly from bronze).

This project uses only the bronze and gold layers: dbt reads directly from bronze and builds the gold star schema and wide table in one step (Section 4.5), since the dataset is small and clean enough that a separate silver cleaning stage would add no value.

Chapter 3

Problem Statement and Challenges

3.1 The Problem Before Streaming Lakehouses

Chapter 2 introduced the pieces a streaming lakehouse is built from: message queues, stream processing engines, open table formats, and medallion layering. None of these are new inventions on their own – what is comparatively recent is combining them into a single pattern. Before that combination existed, teams that wanted both fresh data and a queryable, transactional lake had to work around four gaps, described below.

3.1.1 Latency of Batch ETL

As Section 2.1 already covers, a warehouse fed by scheduled batch loads is always some distance behind reality: dashboards and models reflect the last load, not the current state of the system. This was simply accepted as the cost of running a warehouse-centric stack, because the tools available – scheduled ETL jobs writing into a warehouse – had no continuous alternative.

3.1.2 Lambda-Architecture Complexity

The historical fix for that lag was the *lambda architecture*: a batch layer computing correct, complete results on a schedule, and a separate real-time *speed layer* computing approximate, low-latency results in between batch runs, with the two reconciled downstream. The catch is that the speed layer and the batch layer are usually two different

systems with two different codebases, both implementing the same business logic. Keeping them consistent means writing every transformation twice and manually verifying the two never drift apart – an ongoing operational cost and a recurring source of subtle bugs whenever one side changes and the other does not.

3.1.3 No Serving Layer on the Data Lake

Plain object storage holding raw files (Parquet on S3 or HDFS, without an open table format on top) has no row-level insert, update, or delete, no ACID transactions, and no time travel – it can only store and return whole objects, as Section 2.4 explains. In practice this made the lake a read-only, append-only dump: useful for cheap bulk storage, but unusable as a serving layer for anything that needed a consistent, updatable view of the data. Any workload that needed transactional guarantees had to bypass the lake entirely and go through a separate data warehouse, adding a second copy of the data and another ETL hop to keep it in sync.

3.1.4 Storage and Compute Coupled in the Warehouse

Traditional data warehouses bundle storage and compute into one system, so scaling one means paying for the other whether it is needed or not. Combined with the previous point, this pushed teams into keeping two copies of the same data: a cheap, raw copy in the lake, and a structured, query-ready copy in the warehouse, doubling both storage cost and the pipeline work needed to keep the copies aligned.

Combining a stream processing engine with an open table format (Section 2.4) closes this gap: bronze tables backed by object storage stay cheap, gain COW/MOR transactional writes and upserts, and are fed continuously by a streaming engine rather than on a batch schedule – the *streaming lakehouse* this report builds.

3.2 Technical Challenges

Section 3.1 argued why the streaming-lakehouse paradigm itself is needed. This section is about a different, narrower question: what was concretely hard about building *this*

pipeline on top of that paradigm. Streamify’s data source is Eventsim, which continuously emits synthetic music-streaming events across three Kafka topics – `listen_events`, `page_view_events`, and `auth_events` – rather than arriving in scheduled batches. Building a streaming lakehouse pipeline on top of this dataset surfaces, in a small and reproducible setting, the same distributed-systems problems introduced in Section 2.1: keeping up with a continuous arrival rate, tolerating faults without losing or duplicating data, sharing a fixed compute budget, and handling messy real-world join keys. The rest of this section works through each one as it appears in this project.

3.2.1 Streaming Volume and Velocity

Eventsim continuously emits events across three topics, so the pipeline must ingest, decode, and land data in the lakehouse without falling behind the producer. This requires a message queue that can buffer and parallelize consumption (Kafka, partitioned per topic) and a streaming engine that can keep its micro-batch processing time below the arrival rate.

3.2.2 Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster

Kafka runs as a 2-node KRaft cluster (no ZooKeeper), with each topic replicated across both brokers (replication factor 2, 3 partitions per topic). With only two brokers, the cluster must keep serving every partition if either single broker is restarted or fails, while Spark Structured Streaming must resume exactly where it left off after any restart. This project relies on Structured Streaming’s checkpointed offsets (persisted to a durable checkpoint volume) as the source of truth for resuming the stream, and on Kafka’s per-partition replication across the two brokers for broker-level fault tolerance.

3.2.3 Computing and Partitioning on a Small, Shared Spark Cluster

The Spark Standalone cluster totals only 4 cores across one master and two workers (2 cores / 2 GB each), and those 4 cores are shared between two long-running applications: the Spark Structured Streaming ingestion job (capped at 2 cores) and the Spark Thrift

Server that dbt submits its SQL transformations through (also capped at 2 cores). This leaves no spare capacity for ad-hoc Spark SQL at this cluster size, and means both the streaming job's micro-batch partitioning and dbt's batch transformation jobs must fit within their fixed 2-core budget rather than being able to scale out elastically.

Chapter 4

Proposed Solution

4.1 Architecture Overview

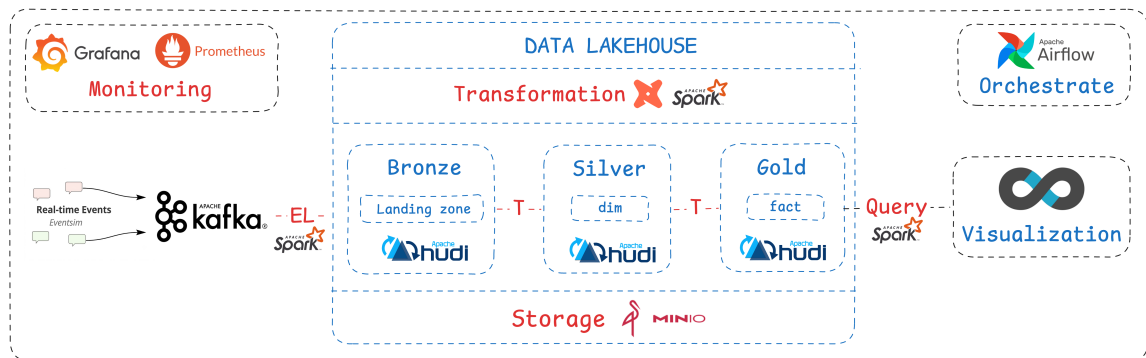


Figure 6: Streamify architecture overview

Figure 6 shows the end-to-end architecture: Eventsim generates events into a Kafka [2] cluster, Spark Structured Streaming [4] consumes and lands them as Hudi [1] bronze tables, dbt transforms the bronze layer into a star schema and a wide table on the same Spark cluster, and Superset visualizes the result through the Spark Thrift Server, alongside a Prometheus/Grafana monitoring stack.

Layer	Tool	Role
Data simulation	Eventsim	Generates synthetic music-streaming events based on the Million Song Dataset
Message queue	Apache Kafka 4.3 (KRaft)	2-node cluster, replication factor 2, 3 partitions per topic, no ZooKeeper
Stream processing	Spark Structured Streaming 3.5.3	Reads Kafka, decodes/enriches events, writes bronze tables to Hudi
Table format / storage	Apache Hudi / Minio	MERGE_ON_READ bronze streaming tables, COPY_ON_WRITE gold/seed tables, on MinIO/S3 via the Hadoop S3A connector
Compute cluster	Spark Standalone (1 master, 2 workers)	Shared by the streaming job and dbt (via Thrift), 4 cores total
Transformation	dbt (dbt-spark, method: thrift)	SQL-based star schema and wide-table modelling, submitted via the Spark Thrift Server
Visualization	Apache Superset	Dashboards connected to the Spark Thrift Server
Monitoring	Prometheus, Grafana	Kafka consumer-group lag, Spark JVM/streaming metrics, per-container CPU/memory
Containerization	Docker Compose	Single-command reproducible deployment

Table 4.1: Technology stack per pipeline layer.

The remainder of this chapter walks through the pipeline one layer at a time – ingestion, stream processing, storage, transformation, and serving – followed by how the whole stack is deployed. Each layer section closes with a short note on why that specific tool was chosen, cross-referencing the fuller alternatives-and-trade-offs discussion in Chapter 2.

4.2 Storage Layer

Tables are Hudi-managed, backed by MinIO [6]/S3 locally or real S3 in aws mode via the Hadoop S3A connector, with no rebuild needed to switch between the two – only an environment variable (LAKE_ENV). There is no separate catalog service: each table is self-describing at its storage path (a `.hoodie/` metadata directory) and is (re-)registered into Spark’s catalog idempotently at job startup. Bronze tables use the `MERGE_ON_READ` table type, so fast streaming appends land as delta log files that are periodically compacted into columnar base files; gold and seed tables use `COPY_ON_WRITE`, rebuilt via full overwrite.

Why Hudi, and why MOR bronze / COW gold. Hudi’s self-describing `.hoodie/` metadata avoids the extra catalog or warehouse service that plain Parquet or Snowflake would require. Bronze uses MOR to keep frequent streaming appends cheap; gold uses COW because it is rebuilt once per run and then read repeatedly by Superset.

4.3 Ingestion Layer

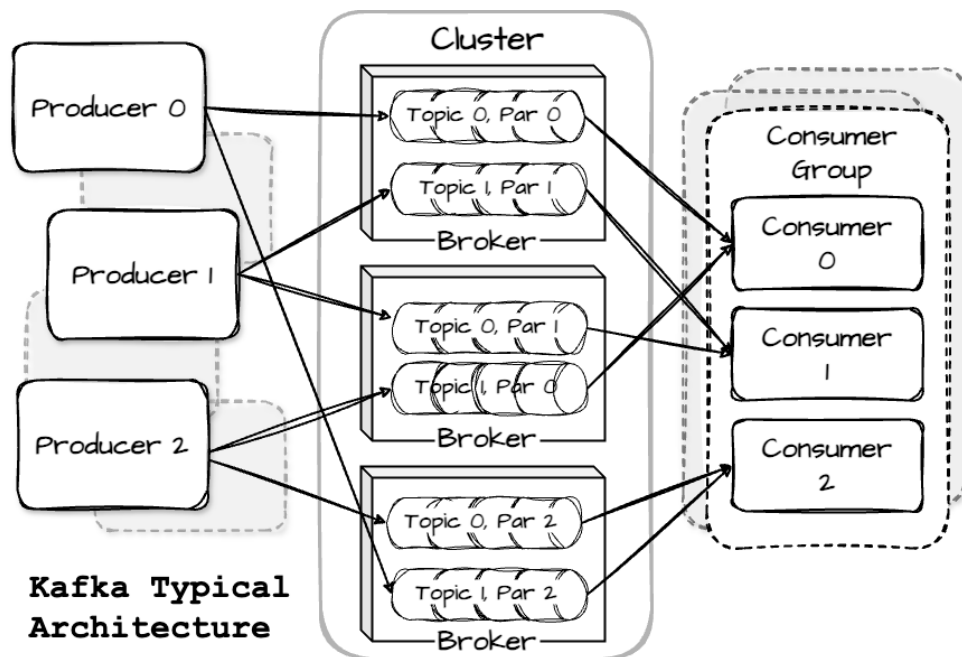


Figure 7: Kafka internal architecture: 2-node KRaft cluster, 3 partitions per topic, replication factor 2.

Eventsim simulates a real-world user base, generating page-view and song-play events that mimic activity on a music-streaming website. Apache Kafka 4.3 runs in KRaft mode (no ZooKeeper) as a 2-node cluster (broker-1, broker-2). Each of the three event topics – `listen_events`, `page_view_events`, `auth_events` – has 3 partitions and replication factor 2, so every partition has a replica on both brokers, as shown in Figure 7.

Because KRaft mode is used, each broker itself also acts as a metadata/controller node, removing the need for a separate ZooKeeper ensemble. For each topic, the 3 partitions are spread across the two brokers, and because replication factor is 2, every partition is fully replicated: one broker holds the leader replica for a given partition and the other holds the follower replica, and the two brokers swap this role across partitions so that neither broker is leader for every partition of a topic. Producers (Eventsim) and consumers (the Spark Structured Streaming job) always read from and write to the leader replica of a partition; the follower replica only pulls data from the leader to stay in sync, and takes over as leader if its broker fails. This is what gives the cluster broker-level fault tolerance despite running on just two nodes.

4.4 Stream & Batch Processing Layer

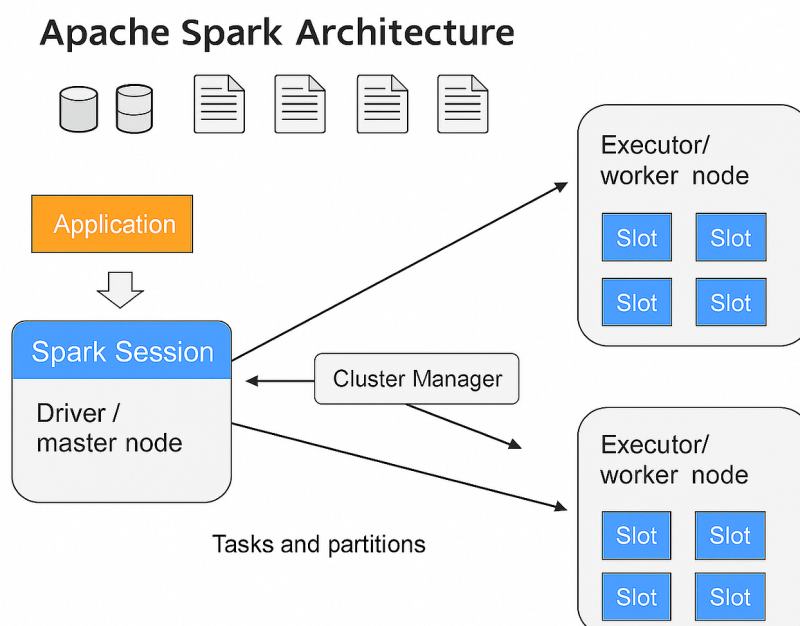


Figure 8: Spark cluster architecture: driver, master, and worker/executor processes.

The Spark Standalone cluster shown in Figure 8 runs both the streaming job and dbt's batch queries. A *driver* process plans a job and schedules its tasks; the *master* tracks which workers are free and grants the driver executor slots on them; each *worker* then launches one or more *executor* processes that run the driver's tasks and cache data. The streaming job's driver is submitted once and keeps its executor slots for the life of the pipeline, while dbt submits short-lived batch queries through the Thrift Server on the same cluster.

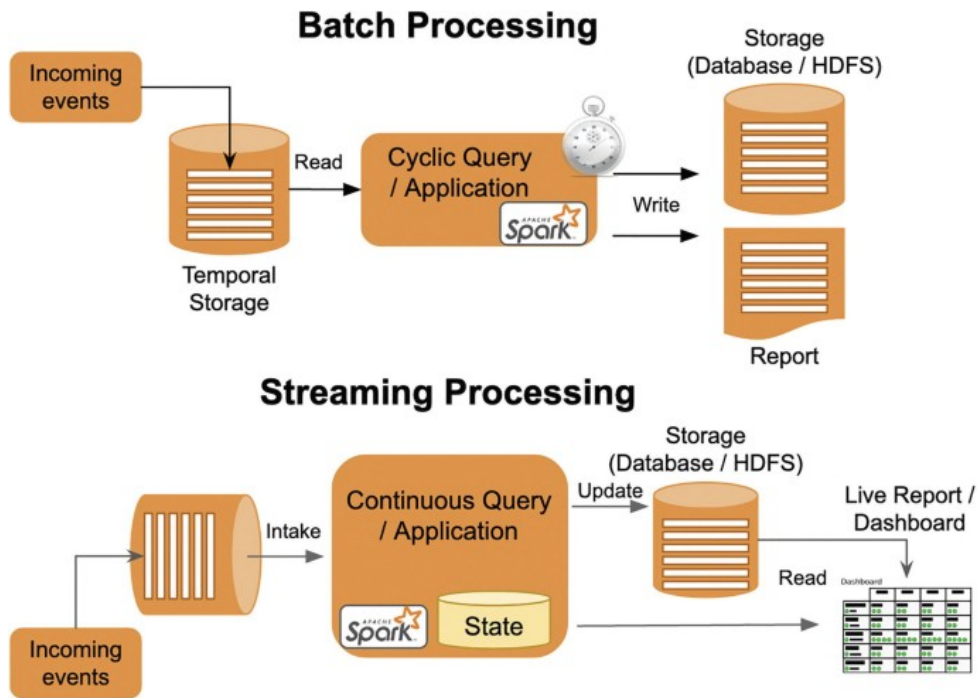


Figure 9: Spark Structured Streaming's micro-batch model: an unbounded stream is sliced into small batches, each run as an ordinary Spark batch job.

Two layers of work share this cluster. The *streaming layer* is the long-running ingestion job: it consumes all three Kafka topics, fixes latin1-mojibake on text fields, derives time columns, and appends each micro-batch to bronze Hudi tables (staging.<topic>). The *batch layer* is everything downstream of bronze – dbt and Superset: dbt runs on the same Spark cluster as the streaming job, reading the bronze Hudi tables and writing the gold Hudi tables, and Superset connects to the Spark Thrift Server to query those gold tables for visualization.

Structured Streaming does not process events one at a time. Instead, as Figure 9 shows, the driver repeatedly polls the three Kafka topics for new offsets, groups the newly available records into a micro-batch, and schedules that micro-batch as a short-

lived Spark job whose tasks run on the worker executors, in the same way a batch job would.

The streaming job runs as a driver program submitted to the Spark Standalone cluster described in Section 4.8 (1 master, 2 workers, 4 cores total, shared with the Thrift Server dbt submits through). Each micro-batch's tasks read their assigned Kafka partitions, decode and enrich the events (`spark_streaming/jobs/stream_events.py`), and append the results to the bronze Hudi tables before the driver commits the checkpoint and moves on to the next batch. Because this job and the Thrift Server share the same 4-core cluster, the size and frequency of micro-batches are bounded by whatever cores are not already claimed by the Thrift Server at that moment.

Structured Streaming resumes from a persisted checkpoint volume (`spark-checkpoints`) on restart; each micro-batch also commits its Kafka offsets to a `streamify-<topic>` consumer group purely so lag is visible in Grafana (Spark itself never depends on that commit to resume).

4.5 Transformation Layer



Figure 10: Star schema: fact_streams and its five dimension tables.

The transformation layer turns each bronze Hudi table into an analytical data model, splitting it into a fact table and its dimension tables so downstream queries never touch bronze directly:

- **Star schema.** One fact table, **fact_streams**, joined to five dimension tables: **dim_users**, **dim_songs**, **dim_artists**, **dim_locations**, and **dim_datetime**. This is the normalized, storage-efficient model (Section 2.4).
- **Wide table (wide_streams).** A denormalized table built from the star schema by pre-joining **fact_streams** with all five dimension tables, so that Superset can query a single flat table instead of performing joins at dashboard-query time.



Figure 11: dbt DAG showing the dependency between bronze sources, the star schema, and `wide_streams`.

Rather than hand-writing this as Spark job code, dbt [5] acts as a SQL abstraction layer above the same Spark cluster: it generates the underlying Spark SQL and submits it through the Spark Thrift Server (method: `thrift`), materializing both model layers as `COPY_ON_WRITE` Hudi tables rebuilt via full overwrite each run. dbt builds a DAG of models – declared in `dbt_project.yml`, with each model a SQL query under `models/` – where every model transforms one or more upstream models into a new table. Figure 11 shows the resulting DAG, with bronze sources at the bottom, the star schema in the middle, and `wide_streams` at the top.

Why dbt, and why both a star schema and a wide table. dbt was chosen over a layer of hand-written Spark SQL scripts because it declares model dependencies explicitly (via `ref()`/`source()`), runs a built-in test framework against every model (uniqueness, not-null, referential integrity), generates documentation from the same source files, and is friendlier to iterate on during development – properties that matter given the five-dimension join the fact table depends on. Modelling both a star schema and a separate wide table, rather than either alone, keeps normalization and storage efficiency in the star schema while keeping BI query simplicity in `wide_streams`, instead of trading one off against the other.

4.6 Serving Layer

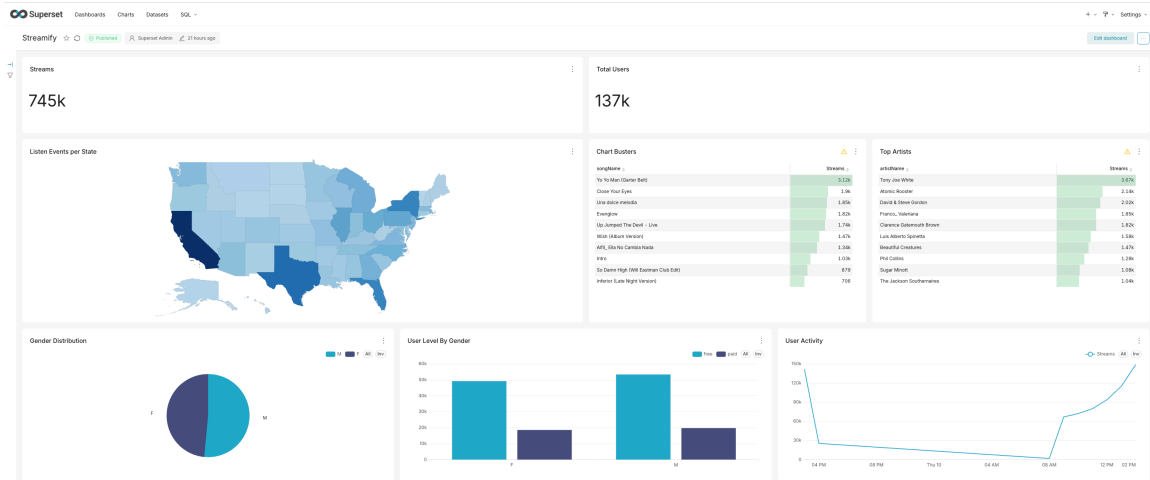


Figure 12: Superset results dashboard, built on top of `wide_streams` and queried through the Spark Thrift Server.

Apache Superset [3] connects to the same Spark Thrift Server that dbt submits SQL through, querying the `wide_streams` model to visualize insights through interactive dashboards (`superset/provision_dashboard.py` provisions them via the REST API). Figure 12 shows the resulting dashboard layout, giving end users a single, pre-joined table to explore without needing to understand the star schema behind it.

Superset ships a native Thrift/HiveServer2-compatible connector that talks directly to the Spark Thrift Server endpoint already used by dbt, needing no extra license or gateway infrastructure, unlike Power BI or Tableau.

4.7 Monitoring Layer

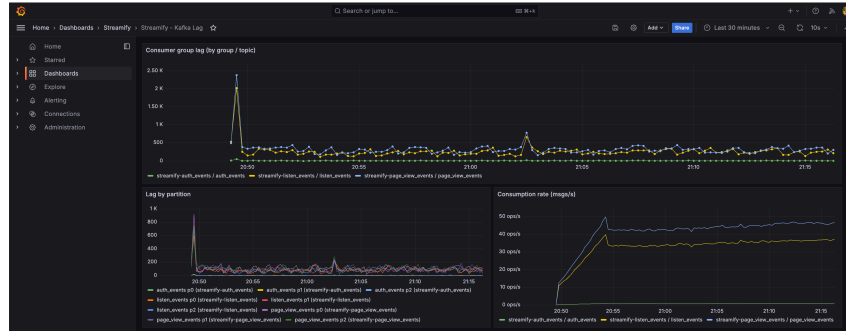


Figure 13: Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.



Figure 14: Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.

Three exporters feed the same Prometheus instance: kafka-exporter scrapes each consumer group’s per-partition lag directly from Kafka, Spark’s built-in metrics endpoint exposes streaming/batch job counters and JVM heap usage, and cAdvisor reports per-container CPU and memory for every service in the stack. Grafana queries Prometheus and renders the two dashboards in Figures 13 and 14: Kafka consumer-group lag and consumption rate per topic, and Spark job/JVM/container metrics. This gives qualitative evidence of the pipeline’s throughput and health under load, rather than a formal benchmark.

4.8 Deployment & Environment

The whole stack is deployed with Docker Compose (`docker-compose.yml`), with a `docker-compose.dev.yml` overlay that adds MinIO for local, S3-free development. Spark runs as a Standalone cluster of 1 master and 2 workers, 2 cores / 2 GB each (4 cores total), shared between the streaming ingestion job and the Spark Thrift Server that dbt submits SQL through, each capped at 2 cores.

Why Spark Standalone at this size. Spark Standalone was chosen over YARN or Kubernetes because both alternatives add a separate resource-management control plane that is not justified when the entire compute budget is 4 cores across one master and two workers; Standalone instead runs as three containers alongside the rest of the pipeline in the same `docker-compose.yml`. The direct consequence is that the streaming job and the Thrift Server, each capped at a fixed 2 cores, together already consume the entire cluster, leaving no headroom for ad-hoc Spark SQL while both are running.

Chapter 5

Conclusion

5.1 Results Achieved

This project achieved its core goal: understanding, first-hand rather than from a diagram, how a modern streaming platform actually works end to end. Building the pipeline meant learning why a Kafka partition needs a follower replica and how that replica takes over on a broker failure, why Structured Streaming’s checkpoint has to be separate from the consumer-group offset it also commits, when `MERGE_ON_READ` is worth its read-time merge cost over `COPY_ON_WRITE`, and how a fixed, shared compute budget forces every downstream decision – bronze table layout, how many dbt models to run, how often a dashboard can refresh – to fit inside that budget rather than assuming elastic scale.

It also achieved the goal of applying the distributed-systems concepts from the course to a real system instead of studying them in the abstract: partitioning and replication, checkpointed exactly-once processing, and consumer-lag observability were each exercised against a real failure mode, not just described. In doing so, the project confirmed in practice why the streaming-lakehouse combination closes the gaps identified in Section 3.1 – no separate lambda-architecture speed layer duplicating logic, a lake that is transactionally updatable instead of a read-only dump, and storage and compute that scale independently – and it did so reproducibly, with the whole stack coming up from a single Docker Compose command.

5.2 Limitations

- **Small, fixed-size cluster.** The Spark Standalone cluster totals only 4 cores, shared between the streaming job and the Thrift Server that dbt uses, leaving no headroom for ad-hoc Spark SQL or additional concurrent workloads.
- **No autoscaling.** Kafka brokers, Spark workers, and other services are fixed at deployment time; the pipeline does not scale resources up or down in response to load.
- **Synthetic dataset.** Eventsim generates realistic-looking but entirely synthetic user activity over a fixed 10,000-song catalog, so the pipeline has not been validated against real production traffic patterns or data-quality issues.

5.3 Future Directions

- Replace or augment the synthetic Eventsim data with a real or real-scale streaming dataset to validate the pipeline under realistic data-quality and volume conditions.
- Add autoscaling for the Kafka and Spark clusters (e.g. via a container orchestrator) so cluster capacity can grow with load instead of being fixed at deployment time.
- Extend monitoring and alerting beyond the current Grafana dashboards, e.g. automated alerts on consumer lag thresholds, streaming job failures, or dbt run failures.
- Add a silver layer between bronze and gold once source schemas or data quality no longer stay clean enough for dbt to model gold directly from bronze.

References

- [1] Apache Software Foundation. *Apache Hudi Documentation*. 2024. URL: <https://hudi.apache.org/docs/overview> (visited on 09/12/2026).
- [2] Apache Software Foundation. *Apache Kafka Documentation*. 2024. URL: <https://kafka.apache.org/documentation/> (visited on 09/12/2026).
- [3] Apache Software Foundation. *Apache Superset Documentation*. 2024. URL: <https://superset.apache.org/docs/intro> (visited on 09/12/2026).
- [4] Apache Software Foundation. *Structured Streaming Programming Guide*. 2024. URL: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (visited on 09/12/2026).
- [5] dbt Labs. *dbt Documentation*. 2024. URL: <https://docs.getdbt.com/> (visited on 09/12/2026).
- [6] MinIO, Inc. *MinIO Documentation*. 2024. URL: <https://min.io/docs/minio/linux/index.html> (visited on 09/12/2026).